

GPU-Based Parallelization of a Shallow Neural Network using Cuda

Mahdi Ladoul¹, Mohamed Ali Hamroun¹, Fares Abderraouf Banziada¹, and Abderrahmane Grine¹

¹4th year Computer Systems Option (SIQ), École nationale supérieure d'informatique, Algiers, Algeria.

Abstract

This paper analyzes a baseline GPU parallelization strategy for training a shallow neural network and proposes an alternative strategy aimed at improving computational performance. The study focuses on optimizing matrix multiplication parallelization and evaluates the impact of different execution models on training time.

1 Reference Parallelization Strategy

Shallow Neural Network Overview

A shallow neural network (SNN) is a feedforward model with a single hidden layer between the input and output layers. Despite its simplicity, it can capture non-linear relationships and is often used as a reference model in performance-focused studies due to its clear computational structure.

In this work, the SNN comprises an input layer, one fully connected hidden layer, and an output layer. The input layer receives fixed-dimension feature vectors that are linearly transformed by the first weight matrix. The hidden layer applies a non-linear activation function to introduce model expressiveness. The output layer then produces predictions through a second linear transformation.

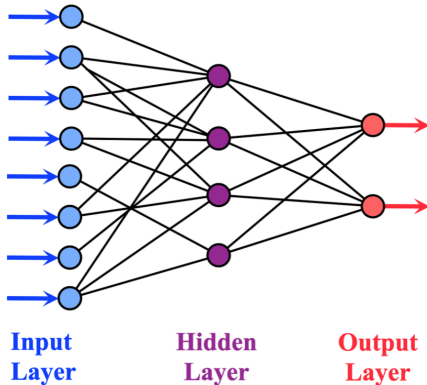


Figure 1: Shallow Neural Network Architecture

Forward propagation is dominated by dense matrix multiplications, and backward propagation relies on matrix multiplications and transpositions. Thus, training time is largely determined by these linear algebra operations.

The shallow network is an ideal case study for GPU parallelization, as its simple architecture highlights computational performance without the complexity of deeper models.

Network Parameters

The shallow neural network used as the baseline implementation consists of a single hidden layer between the input and output layers. Its architecture and training parameters are chosen to ensure reproducibility and enable consistent performance evaluation across different parallelization strategies.

Network Architecture Parameters

Table 1: Model Parameters

Parameter	Value
INPUT_SIZE	32 features per sample
HIDDEN_SIZE	256 neurons
OUTPUT_SIZE	1 neuron

The network consists of a single fully connected hidden layer, making it a shallow neural network. This configuration was chosen to keep the computational structure simple while preserving a sufficiently large

workload for performance evaluation.

Training Parameters

Table 2: Training Hyperparameters

Parameter	Value
EPOCHS	100
BATCH_SIZE	256
LEARNING_RATE	0.002
Loss function	Mean Squared Error (MSE)
Optimization method	Mini-batch gradient descent

Mini-batch training is employed to balance computational efficiency and convergence stability. The batch size is selected to generate matrix dimensions that are large enough to benefit from GPU parallel execution while remaining compatible with available memory resources.

GPU Execution Parameters

Table 3: Kernel Configuration

Parameter	Value
THREADS_PER_BLOCK	16

Algorithm Flow (Forward & Backward Pass)

let $X \in \mathbb{R}^{m \times n}$ denote a mini-batch of input samples, where m is the batch size and $n = 32$ is the number of features per sample. Let $W_1 \in \mathbb{R}^{n \times h}$ and $W_2 \in \mathbb{R}^{h \times o}$ represent the weight matrices of the hidden and output layers, respectively, with $h = 256$ and neurons in the hidden layer and $o = 1$ output neuron.

Algorithm 1 Forward Propagation

- 1: **Input:** X, Y, W_1, W_2
- 2: **Output:** $\hat{Y}$, loss $\mathcal{L}$
- 3: Compute hidden layer pre-activation:
- 4: $Z_1 \leftarrow XW_1$
- 5: Apply ReLU activation:
- 6: $A_1 \leftarrow \max(0, Z_1)$
- 7: Compute output layer pre-activation:
- 8: $\hat{Y} \leftarrow A_1W_2$
- 9: Compute Mean Squared Error loss:
- 10: $\mathcal{L} \leftarrow \frac{1}{m} \sum_{i=1}^m (\hat{Y}_i - Y_i)^2$

Mini-Batch Processing The forward and backward steps are applied iteratively for each mini-batch

Algorithm 2 Backward Propagation

- 1: **Input:** $X, Y, A_1, Z_1, \hat{Y}, W_1, W_2$, learning rate η
- 2: **Output:** Updated weights W_1, W_2
- 3: Compute output layer gradient:
- 4: $dZ_2 \leftarrow \frac{2}{m}(\hat{Y} - Y)$
- 5: Compute gradient w.r.t. output weights:
- 6: $dW_2 \leftarrow A_1^\top dZ_2$
- 7: Backpropagate to hidden layer:
- 8: $dZ_1 \leftarrow dZ_2W_2^\top$
- 9: Apply ReLU derivative:
- 10: $dZ_1 \leftarrow dZ_1 \odot \mathbb{K}_{Z_1 > 0}$
- 11: Compute gradient w.r.t. hidden weights:
- 12: $dW_1 \leftarrow X^\top dZ_1$
- 13: Update weights using gradient descent:
- 14: $W_1 \leftarrow W_1 - \eta dW_1$
- 15: $W_2 \leftarrow W_2 - \eta dW_2$

of size $m=256$. The training loop repeats these operations for 100 epochs, ensuring convergence of the network parameters toward minimizing the MSE loss.

CUDA-Based Parallelization Strategy

The CUDA implementation parallelizes matrix multiplication by offloading computations to the GPU. For each multiplication, input and output matrices are copied from host to device memory, and the result is copied back after computation.

Each thread computes a single output element, with threads organized in 16×16 blocks and mapped to a two-dimensional grid covering the output matrix. The kernel performs standard matrix multiplication using global memory, where each thread iterates over the shared dimension to compute its dot product. Device synchronization ensures completion before returning results to the host.

Granularity and Synchronization

The baseline CUDA parallelization assigns one thread per output element in 16×16 blocks, exposing high parallelism but with no intra-block data sharing. Each thread redundantly loads input data from global memory, making memory access inefficient. Minimal kernel synchronization is needed, but `cudaDeviceSynchronize()` is called after each launch, and forward/backward multiplications are executed sequentially, preventing overlap of computation and memory transfers.

Limitations of the Baseline Strategy

Although the baseline implementation is functionally correct and exposes parallelism, it suffers from performance limitations related to memory usage, computation granularity, and synchronization. As a result, GPU resources are underutilized, especially for moderate matrix sizes.

The baseline relies solely on global memory, repeatedly transferring data between host and device and allocating memory for every matrix multiplication. Threads redundantly load overlapping data, resulting in low arithmetic intensity and a memory-wall bottleneck, where computation stalls waiting for memory accesses.

Global synchronization after each kernel launch prevents overlapping computation with data transfers, enforcing sequential execution. This not only reduces throughput but also leaves opportunities for multi-streaming, where concurrent data transfers and kernel execution could hide memory latency and improve utilization.

Table 4: Memory Behavior Analysis

Aspect	Baseline behavior	Impact on performance
Memory allocation	<code>cudaMalloc</code> / <code>cudaFree</code> per operation	High allocation overhead
Memory transfers	H2D and D2H at every multiplication	Significant latency
Memory hierarchy	Global memory only	Low data reuse
Shared memory	Not used	Inefficient bandwidth usage
Synchronization	After each kernel	No overlap of execution
Kernel design	One operation per kernel	Kernel launch overhead

Execution Time Results

Experimental Setup

The reference parallel GPU-based training strategy was evaluated on a single Tesla T4 GPU.

Table 5: Specifications of the Tesla T4 GPU.

Feature	Specification
Device	Tesla T4
Compute capability	7.5
Total global memory	14,912 MB
Multiprocessor count	40
Max threads per block	1024
Max thread dimensions	$1024 \times 1024 \times 64$
Max grid size	$2,147,483,647 \times 65,535 \times 65,535$

The results of the performance evaluation of the shallow neural network training across different datasets. Execution times (in seconds) are summarized in the following table.

Table 6: CUDA Execution Time vs. Thread Configuration

Threads	Small	Medium	Large
4	0.60	3.30	31.91
8	0.57	3.21	31.80
16	0.61	3.23	31.72
32	0.70	3.37	33.11

2 Alternative parallelization strategy

Overview

We propose a GPU-centric parallelization strategy for training the neural network, designed to maximize throughput while minimizing memory overhead. The approach is based on four core principles:

1. Tiled matrix multiplication using shared memory.
2. Persistent GPU memory allocation.
3. Full GPU execution of forward and backward passes with specialized kernels.
4. Multi-Streaming Optimization.

This design minimizes host-device communication, avoids repeated memory allocation, and exploits fine-grained data parallelism across thousands of GPU threads.

Algorithmic Rationale

Tiled Matrix Multiplication

Matrix multiplication dominates both forward and backward passes. A naive approach repeatedly accesses global memory, creating a bandwidth bottleneck.

We partition matrices into $T \times T$ tiles, loading each tile into shared memory. Threads compute partial products using the local tiles before moving to the next. This increases **arithmetic intensity**, reuses data, and reduces global memory traffic without altering the algorithmic complexity.

Algorithm 3 Tiled Matrix Multiplication Algorithm

```

1: for each tile  $t$  in  $0..[A_{\text{cols}}/T]$  do
2:   Load tile.A from  $A[\text{row}, t \cdot T : (t + 1) \cdot T]$ 
3:   Load tile.B from  $B[t \cdot T : (t + 1) \cdot T, \text{col}]$ 
4:   Synchronize threads
5:   for  $k = 0..T - 1$  do
6:      $value \quad += \quad \text{tile.A}[\text{thread\_row}, k] \cdot$ 
        $\text{tile.B}[k, \text{thread\_col}]$ 
7:   end for
8:   Synchronize threads
9: end for
10:  $C[\text{row}, \text{col}] = value$ 

```

Persistent GPU Memory

Rather than allocating and freeing device memory for each batch, all buffers—weights, activations, gradients, and batch data—are allocated once and remain resident on the GPU.

This eliminates repeated `cudaMalloc/cudaFree` overhead and minimizes PCIe transfers. Only batch inputs are copied from host to device, significantly improving training efficiency.

Algorithm 4 Memory Allocation

```

1: Allocate d.W1, d.W2 {Persistent weights}
2: Allocate d.X_batch, d.Y_batch {Batch buffers}
3: Allocate d.Z1, d.Y_pred {Activations}
4: Allocate d.dZ1, d.dZ2 {Gradients}
5: Allocate d.dW1, d.dW2 {Weight gradients}

```

Fully GPU-Resident Forward and Backward Passes

All operations, including matrix multiplication, activations, gradient computations, and weight updates, are executed entirely on the GPU.

By mapping each element or activation to a thread, we expose fine-grained parallelism that scales with batch size and layer width. This approach reduces synchronization points and keeps intermediate results in device memory, avoiding costly host-device transfers.

Algorithm 5 Fully GPU Passes

```

1: Copy initial weights  $W_1, W_2$  to GPU
2: for epoch = 1 to EPOCHS do
3:   for each batch do
4:     Copy batch X_batch, Y_batch from CPU to GPU
5:     Forward pass entirely on GPU
6:     Compute loss on GPU
7:     Backward pass entirely on GPU
8:     Synchronize GPU
9:   end for
10: end for
11: Free persistent GPU memory

```

Multi-Streaming Optimization

Although most computations remain on the GPU, mini-batch data must still be transferred between host and device. To reduce the impact of these transfers, multiple CUDA streams are used to overlap communication and computation.

Instead of executing all operations in a single stream, data transfer for the next batch is issued asynchronously while the current batch is processed by the forward and backward kernels. This reorganizes the training loop into a pipeline without changing the underlying

Algorithm 6 Overlapped Batch Processing with Streams

```

1: for each batch  $i$  do
2:   Asynchronously copy batch $[i + 1]$  to device using stream B
3:   Launch forward and backward kernels for batch $[i]$  using stream A
4:   Asynchronously copy metrics $[i]$  to host
5:   Synchronize when dependencies arise
6: end for

```

Architectural Justification

- **Thread-level parallelism:** Each matrix element or activation is mapped to a thread, maximizing occupancy and throughput across streaming multiprocessors.
- **Memory hierarchy utilization:** Shared memory stores tiles to reduce global memory accesses, improving effective bandwidth and arithmetic intensity.

- **Minimized host-device communication:** Weights, gradients, and intermediate activations remain on the GPU throughout training. Only batch inputs and occasional loss values are transferred.
- **Efficient synchronization:** Only block-level synchronization (`__syncthreads()`) is used within kernels; global synchronization occurs only at kernel boundaries.
- Modern GPUs support concurrent kernel execution and asynchronous memory transfers across streams. Exploiting this capability provides: **Latency hiding** and **Better utilization**.

Implementation and Experimental Results

Experimental Setup

The proposed parallel GPU-based training strategy was evaluated on a single Tesla T4 GPU, the same as the baseline one.

Verification of Functional Correctness

To verify that the GPU-optimized strategy produces correct results, we compared its outputs to a reference CPU implementation:

To verify that the GPU-optimized strategy produces correct results, we compared its outputs to a reference CPU implementation:

1. **Forward Pass Validation:** Predictions produced by the GPU-based network were compared element-wise with the CPU implementation. Mean absolute differences were below 10^{-1} for all datasets.
2. **Backward Pass Validation:** Gradients computed on the GPU were checked against the CPU reference. Maximum relative error was below 0.01%, indicating numerically consistent updates.

These verifications demonstrate that the parallel GPU strategy is **functionally correct** while achieving substantial speedup.

3 Performance Evaluation and Discussion

Evaluation metrics

The performance of the proposed strategy was assessed using both computational and learning-quality metrics. Execution time (T), measured in seconds, was used to evaluate runtime efficiency.

Performance improvement over the reference implementation was quantified using the speedup metric:

$$S = \frac{T_{\text{reference}}}{T_{\text{proposed}}} \quad (1)$$

In addition, the Mean Squared Error (MSE) was used to assess convergence quality:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2)$$

These metrics ensure that computational acceleration is achieved without compromising learning accuracy.

Influence of Tile size

In order to determine the optimal tiling configuration, the proposed tiled GPU implementation was evaluated on the *large dataset* with 256 hidden neurons. Tile sizes of 4, 8, 16, and 32 were experimentally tested.

Table 7: Tile size.

Tile Size	Execution Time (s)	MSE
4	2.0023	30.552460
8	1.2652	7.364520
16	1.1760	16.705551
32	1.8301	11.864802

The results indicate that a tile size of 8 provides the best trade-off between execution time and convergence quality, achieving a substantial reduction in runtime while yielding the lowest MSE.

Scalability with Respect to Dataset Size

The training execution time was measured for increasing dataset sizes in order to evaluate scalability.

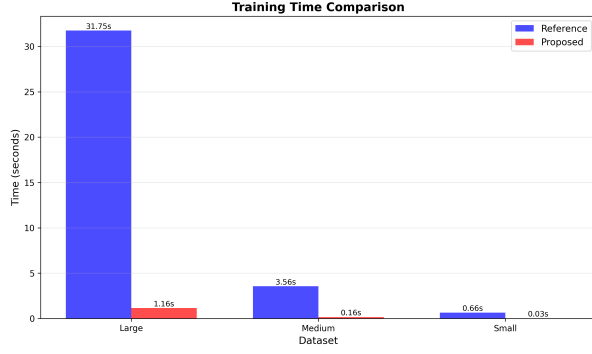


Figure 2: Execution time as a function of dataset size

As expected, execution time increases with dataset size for both the reference and proposed strategies. However, the proposed implementation outperforms the reference approach. The performance gap becomes more pronounced as the dataset size increases, demonstrating improved scalability under higher computational loads.

Speedup Analysis

The acceleration achieved by the proposed strategy is quantified using the speedup metric defined above.

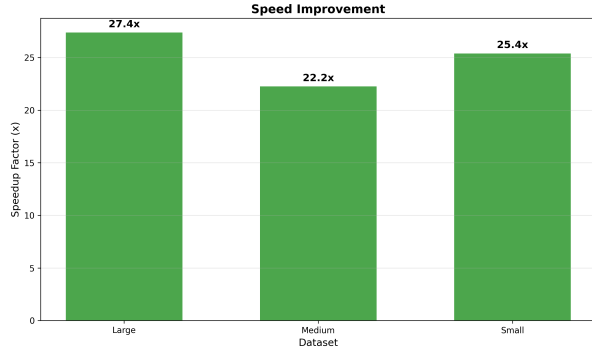


Figure 3: Speedup as a function of dataset size

The results show that the proposed strategy achieves increasing speedup as dataset size grows. This behavior confirms that the GPU implementation effectively exploits parallelism and amortizes memory transfer overhead when computational intensity increases.

Impact of Neurons Number in Hidden Layer

To further evaluate performance scalability, experiments were conducted by varying the number of neurons in the hidden layer.

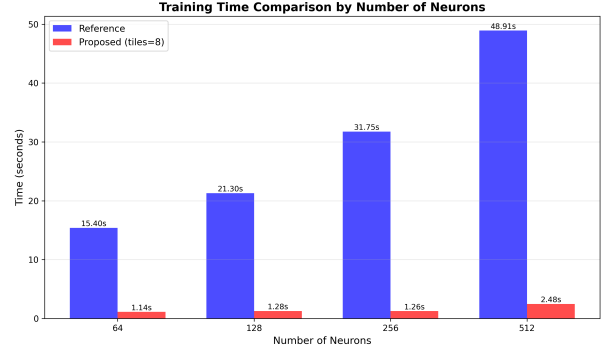


Figure 4: Execution time as a function of number of neurons in the hidden layer

Increasing the number of neurons leads to a proportional growth in computational complexity due to larger number of matrix multiplications. While both strategies exhibit increased execution time, the proposed implementation demonstrates superior scalability. The speedup improves as the network size increases, indicating better utilization of GPU resources under higher arithmetic intensity.

Limitations

Although the proposed GPU strategy demonstrates substantial performance improvements, several limitations should be acknowledged:

- The evaluation was conducted on a single GPU architecture; performance may vary across devices due to differences in memory bandwidth, shared memory size, and compute capability.
- While the implementation mitigates memory wall effects, global memory bandwidth remains a limiting factor for very large workloads. Further optimizations, such as asynchronous memory transfers, were not explored.
- Experiments were limited to a single hidden-layer neural network; the effectiveness of the proposed optimizations on more complex architectures is to be investigated.

4 Conclusion

This work proposes an optimized GPU-based training strategy for shallow neural networks through parallelization optimization using CUDA, that overcomes the main performance limitations of the reference implementation.

The approach addresses memory wall effects, reduces host-device transfer overhead, explore multi-streaming to overlap computation and communication, and minimizes allocation-freeing costs through memory space reuse.

Experimental results demonstrate significant reductions in execution time and increasing speedup as dataset size and network complexity grow. The selected configuration achieves a favorable balance between computational efficiency and numerical stability.

Acknowledgment

We sincerely thank Professor HAICHOIR for her guidance and support throughout this project. Her course provided us with the essential theoretical and practical foundations, and her advice throughout the semester, greatly contributed to the completion of this work.

References

- Haichour, A. S. (2026). Code optimization for nvidia gpus [HPC Course material, École Supérieure d’Informatique (ESI)].
- Brouthen, K., & Akeb, A. (2025). *Exploring parallelization of shallow neural network using cuda* (High Performance Computing Lab report).
- NVIDIA Corporation. (2024). *Cuda c programming guide*. NVIDIA. Retrieved February 12, 2026, from <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Anthropic. (2026). *Claude ai*. <https://claude.ai>